

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №2

Реализация микросервисов

Выполнила:

Маркозубова Анастасия

К3339

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

1 Задача

Опираясь на документ, сформированный в рамках ДЗ4, необходимо реализовать разделение backend-приложения сайта поиска работы на микросервисы. Реализация должна соответствовать спроектированной архитектуре database-per-service: каждый сервис отвечает за собственный домен, работает со своей базой данных и взаимодействует с другими сервисами через HTTP API.

2 Ход работы

В качестве исходной системы использовалось монолитное Express + TypeORM приложение из первой лабораторной работы. В рамках ЛР2 монолит был разделен на шесть сервисов, которые соответствуют границам, описанным в ДЗ4: Auth Service, Profile Service, Resume Service, Vacancy Service, Application Service и Reference Service.

Состав реализованных сервисов:

Сервис	Порт	База данных	Назначение
Auth Service	8001	auth_db	Регистрация, вход, выпуск JWT, хранение пользователей.
Profile Service	8002	profile_db	Профили соискателей и компаний.
Resume Service	8003	resume_db	Резюме, образование и опыт работы.
Vacancy Service	8004	vacancy_db	Вакансии, поиск вакансий и фасад создания отклика.
Application Service	8005	application_db	Отклики на вакансии и изменение статусов откликов.
Reference Service	8006	reference_db	Справочники отраслей и специализаций.



Рисунок 1 - Реализованное разделение backend-приложения на микросервисы

2.1 Разделение базы данных

Прямые связи между таблицами разных доменов были заменены на хранение внешних идентификаторов. Например, Resume Service хранит `seeker_id` и `specialization_id`, но не имеет ORM-связи с таблицами `seekers` и `specializations`. Vacancy Service хранит `company_id` и `specialization_id`, а Application Service хранит `resume_id` и `vacancy_id`.

База	Таблицы	Особенности реализации
auth_db	users	Оставлен UserSubscriber для хеширования паролей.
profile_db	seekers, companies	Профили содержат user_id вместо связи с users.
resume_db	resumes, educations, experiences	Резюме содержит seeker_id и specialization_id.
vacancy_db	vacancies	Вакансия содержит company_id и specialization_id.
application_db	applications	Отклик содержит resume_id и vacancy_id.
reference_db	industries, specializations	Справочник специализаций связан только со своей таблицей industries.

2.2 Настройка сервисов в коде

Для выбора набора контроллеров и сущностей был добавлен конфигурационный слой `service-config.ts`. При запуске сервис читает переменную `SERVICE_NAME` и подключает только те контроллеры и Entity, которые относятся к выбранному домену. Общий `data-source.ts` использует этот набор Entity, поэтому сервис не создает таблицы чужого домена.

Были добавлены конфигурационные файлы окружения:

- `.env.auth`
- `.env.profile`
- `.env.resume`
- `.env.vacancy`
- `.env.application`
- `.env.reference`

Для запуска сервисов добавлены `npm`-команды:

```
npm run start:auth
npm run start:profile
npm run start:resume
npm run start:vacancy
npm run start:application
npm run start:reference
```

Чтобы не запускать шесть окон вручную, реализован PowerShell-скрипт `scripts/start-services.ps1`. Он создает базы данных через `scripts/create-databases.ps1`, выполняет сборку TypeScript и открывает отдельное окно терминала для каждого сервиса.

```
npm run db:create
npm run start:services
```

2.3 Межсервисное взаимодействие

Взаимодействие между сервисами реализовано через HTTP-запросы к внутренним маршрутам `/internal/...`. Для этого добавлен общий helper `service-client.ts`, который обращается к соседним сервисам, обрабатывает неуспешные ответы и преобразует их в `HttpError`.

Сервис-источник	Внутренний запрос	Назначение
Profile	GET <code>/auth/internal/users/{id}</code>	Проверка существования пользователя при создании профиля.
Resume	GET <code>/seekers/internal/by-user/{userId}</code>	Получение профиля соискателя для создания резюме.
Resume	GET <code>/specializations/internal/{id}</code>	Проверка специализации.
Vacancy	GET <code>/companies/internal/by-user/{userId}</code>	Получение компании работодателя.
Vacancy	POST <code>/applications/internal</code>	Создание отклика через Application Service.

Application	GET /resumes/internal/{id}/ownership	Проверка владельца резюме.
Application	GET /vacancies/internal/{id}/ownership	Проверка владельца вакансии.

2.4 Документация и Swagger

Для каждого сервиса доступна отдельная Swagger-документация по адресу /docs. При генерации OpenAPI была добавлена фильтрация маршрутов: каждый сервис показывает только свои endpoint'ы, а не все контроллеры проекта. Это важно, так как исходный проект хранит все контроллеры в одном репозитории, но фактически запускает их разными сервисами.

Сервис	Swagger
Auth Service	http://localhost:8001/docs
Profile Service	http://localhost:8002/docs
Resume Service	http://localhost:8003/docs
Vacancy Service	http://localhost:8004/docs
Application Service	http://localhost:8005/docs
Reference Service	http://localhost:8006/docs

2.5 Проверка результата

После реализации была выполнена сборка проекта:

```
npm run build
```

Сборка TypeScript завершилась успешно. Также были созданы базы auth_db, profile_db, resume_db, vacancy_db, application_db и reference_db. После запуска npm run start:services все сервисы начали слушать порты 8001-8006. Дополнительно была проверена работа справочного сервиса через запрос GET /api/industries и доступность Swagger-документации.

Проверка	Результат
npm run build	Успешная сборка без ошибок TypeScript.
npm run db:create	Созданы или подтверждены шесть отдельных PostgreSQL-баз.
npm run start:services	Запущены шесть сервисов в отдельных терминалах.
GET http://localhost:8006/api/industries	Возвращается список отраслей из Reference Service.
http://localhost:8004/docs	Swagger Vacancy Service показывает только маршруты вакансий и связанные внутренние маршруты.

3 Вывод

В результате лабораторной работы монолитное backend-приложение сайта поиска работы было практически разделено на микросервисы в соответствии с техническим дизайном из Д34.

Каждый сервис получил собственную зону ответственности, отдельную базу данных, отдельный порт запуска и собственную Swagger-документацию.

Прямые связи между доменами были заменены хранением идентификаторов и межсервисными HTTP-запросами. Такой вариант соответствует подходу database-per-service и позволяет независимо развивать Auth, Profile, Resume, Vacancy, Application и Reference Service. Для удобства проверки были добавлены скрипты автоматического создания баз данных и запуска всех сервисов одной командой.